Part 2a

I measured the software overhead of maintaining the extent-based page table by comparing the modified kernel against the baseline page fault handler without extent tracking. The benchmark used a private anonymous mmap region, touched one 4 KB page at a time, and disabled transparent huge pages with MADV_NOHUGEPAGE so that each first touch would correspond closely to a normal anonymous page fault. I collected perf stat measurements for page faults, minor faults, major faults, task-clock, cycles, and instructions. In both kernels, the workload produced about 262K page faults, all of which were minor faults, with zero major faults, confirming that this was a first-touch anonymous memory workload and not disk-backed paging. Using the steady-state runs, the baseline kernel averaged about 472.33 ms of task-clock, while the extent-enabled kernel averaged about 501.85 ms, which corresponds to an additional overhead of about 6.25%.

2b

I estimated the potential TLB savings if a single TLB entry could map an entire extent instead of only one 4 KB page. In the single-threaded write benchmark with 65,536 pages touched, the process exited with 2,171 extents on the extent-enabled kernel. This means that, instead of requiring approximately 65,536 TLB entries in a page-based design, an extent-based design would require only about 2,171 entries for the same region. Therefore, the number of saved TLB entries would be 65,536 minus 2,171, which is 63,365 saved entries, or about a 96.69% reduction. This demonstrates the main benefit of the design: although the software page fault path becomes more expensive, the number of translation groups drops a lot when physically contiguous pages are turned into extents.